

DATABASE DESIGN

AI-Powered IT Service Desk & Incident Management System

Version 1.0 — Initial Database Structure

Status: Draft / Under Review

Program	MP Online Internship Program
Project Type	C# / ASP.NET Core
Document	Database Design
Team Size	7 Members
Version	1.0 — Initial Database Structure
Date	August 2026
Status	Draft / Under Review

1. Introduction

The Database Design document defines the initial structure of the database for the AI-Powered IT Service Desk & Incident Management System.

The Requirement Analysis established the business problem, stakeholders, actors, functional requirements, AI-assisted requirements, automation requirements, and non-functional requirements of the system. The database design is the next step in converting those requirements into a structured data model.

This Version 1 database structure is based on the database model currently prepared by the project team.

The purpose of this version is to establish the initial relational database structure, identify the major entities and relationships, and provide a foundation for the next stages of System Design and Development.

IMPORTANT

Version 1 is an initial database structure and is not considered the final production database design. Additional tables, fields, constraints, and relationships may be introduced after System Design and detailed requirement-to-database validation.

2. Database Design Objectives

The initial database is designed to support the major functionality identified in the Requirement Analysis:

- User and role management
- Incident creation and tracking
- Incident categorization
- Incident assignment
- Employee–support communication
- AI-assisted incident analysis
- Notifications
- Incident escalation
- Incident history and resolution

The Requirement Analysis identifies incident creation, assignment, status management, communication, AI analysis, notifications, and escalation as important system functionality.

3. Version 1 Database Structure

Version 1 contains the following 9 tables:

- 1. Roles
- 2. Users
- 3. Categories
- 4. Incidents
- 5. IncidentAssignments
- 6. IncidentComments

- 7. AIAnalysis
- 8. Notifications
- 9. Escalations

The Incidents table acts as the central entity around which most incident-management functionality is organized.

4. Entity Relationship Overview

The major relationship structure is:

Roles → Users → Incidents

The Incidents table is connected to: Categories, IncidentAssignments, IncidentComments, AIAnalysis, Notifications, and Escalations.

Simplified Structure



5. Table Design

5.1 Roles

The Roles table stores the roles available within the system.

Column	Data Type	Constraint	Description
RoleId	INT	Primary Key, Identity	Unique role identifier
RoleName	VARCHAR(50)	NOT NULL	Name of the role

Foreign Key

`Roles.RoleId → Users.RoleId`

Purpose

One role can be associated with multiple users.

The Requirement Analysis identifies four human actors: Employee, Help Desk Agent, IT Administrator, and IT Manager.

5.2 Users

The Users table stores users who access the system.

Column	Data Type	Constraint	Description
UserId	INT	Primary Key, Identity	Unique user identifier
Name	VARCHAR(100)	NOT NULL	User name
Email	VARCHAR(150)	NOT NULL, UNIQUE	User email address
PasswordHash	VARCHAR(255)	NOT NULL	Password hash
RoleId	INT	NOT NULL, Foreign Key	Assigned role

Foreign Key

`Users.RoleId → Roles.RoleId`

Purpose

This table supports authentication, role assignment, incident creation, incident assignment, comments, notifications, and escalation.

The Requirement Analysis requires authenticated users and role-based access to protected functionality.

5.3 Categories

The Categories table stores incident categories.

Column	Data Type	Constraint	Description
CategoryId	INT	Primary Key, Identity	Unique category identifier
CategoryName	VARCHAR(100)	NOT NULL, UNIQUE	Category name

Purpose

Categories allow incidents to be organized and classified consistently.

The Administrator is required to manage incident categories.

5.4 Incidents (Central Table)

The Incidents table is the central table in the Version 1 database.

Column	Data Type	Constraint	Description
IncidentId	INT	Primary Key, Identity	Unique incident identifier
Title	VARCHAR(200)	NOT NULL	Incident title
Description	VARCHAR(MAX)	NOT NULL	Detailed incident description
CategoryId	INT	NOT NULL, Foreign Key	Incident category
Priority	VARCHAR(50)	NOT NULL	Incident priority
Status	VARCHAR(50)	NOT NULL, Default 'Open'	Current incident status
CreatedBy	INT	NOT NULL, Foreign Key	User who created incident
CreatedAt	DATETIME	NOT NULL, Default GETDATE()	Creation date/time
ResolvedAt	DATETIME	NULL	Resolution date/time

Foreign Keys

`Incidents.CategoryId` → `Categories.CategoryId`

`Incidents.CreatedBy` → `Users.UserId`

Purpose

This table stores the primary information about an IT incident.

The Requirement Analysis requires employees to create incidents, provide titles and descriptions, view submitted incidents, and track their status.

The initial status is defined as Open, which is consistent with the acceptance criteria for incident creation.

5.5 IncidentAssignments

The IncidentAssignments table stores information about assigning incidents to support personnel.

Column	Data Type	Constraint	Description
AssignmentId	INT	Primary Key, Identity	Unique assignment identifier
IncidentId	INT	NOT NULL, Foreign Key	Incident being assigned
AssignedTo	INT	NOT NULL, Foreign Key	User assigned to incident
AssignedAt	DATETIME	NOT NULL, Default GETDATE()	Assignment date/time

Foreign Keys

`IncidentAssignments.IncidentId → Incidents.IncidentId`

`IncidentAssignments.AssignedTo → Users.UserId`

Purpose

This supports incident assignment and reassignment by authorized support personnel.

The Requirement Analysis specifies that Help Desk Agents should be able to assign or reassign incidents where authorized.

5.6 IncidentComments

The IncidentComments table stores communication related to incidents.

Column	Data Type	Constraint	Description
CommentId	INT	Primary Key, Identity	Unique comment identifier
IncidentId	INT	NOT NULL, Foreign Key	Related incident
UserId	INT	NOT NULL, Foreign Key	User who created comment
CommentText	VARCHAR(MAX)	NOT NULL	Comment / message
CreatedAt	DATETIME	NOT NULL, Default GETDATE()	Comment date/time

Foreign Keys

`IncidentComments.IncidentId → Incidents.IncidentId`

`IncidentComments.UserId → Users.UserId`

Purpose

This table supports communication between employees and support agents.

The Requirement Analysis includes both employee-to-agent and agent-to-employee communication requirements.

5.7 AIAnalysis

The AIAnalysis table stores AI-generated analysis associated with an incident.

Column	Data Type	Constraint	Description
AIAnalysisId	INT	Primary Key, Identity	Unique analysis identifier
IncidentId	INT	NOT NULL, Foreign Key	Incident analyzed
SuggestedCategory	VARCHAR(100)	NULL	AI-suggested category
SuggestedPriority	VARCHAR(50)	NULL	AI-suggested priority
SuggestedResolution	VARCHAR(MAX)	NULL	AI-suggested resolution
RelatedIncidentId	INT	NULL, Foreign Key	Related / duplicate incident
ConfidenceScore	DECIMAL(5,2)	NULL	AI confidence score
CreatedAt	DATETIME	NOT NULL, Default GETDATE()	Analysis date/time

Foreign Keys

AIAnalysis.IncidentId → Incidents.IncidentId

AIAnalysis.RelatedIncidentId → Incidents.IncidentId

Purpose

The table supports the AI requirements for:

- Category suggestions
- Priority / severity suggestions
- Suggested resolutions
- Related or duplicate incident detection
- Confidence scoring

These capabilities are explicitly defined in the Requirement Analysis.

5.8 Notifications

The Notifications table stores notifications generated for users.

Column	Data Type	Constraint	Description
NotificationId	INT	Primary Key, Identity	Unique notification identifier
UserId	INT	NOT NULL, Foreign Key	Notification recipient
IncidentId	INT	NULL, Foreign Key	Related incident
Message	VARCHAR(500)	NOT NULL	Notification message
IsRead	BIT	NOT NULL, Default 0	Read / unread state
CreatedAt	DATETIME	NOT NULL, Default GETDATE()	Notification date/time

Foreign Keys

`Notifications.UserId → Users.UserId`

`Notifications.IncidentId → Incidents.IncidentId`

Purpose

Notifications support important incident events such as assignment, status changes, and critical incidents.

The Requirement Analysis specifies automated notifications for these events.

5.9 Escalations

The Escalations table stores incident escalation information.

Column	Data Type	Constraint	Description
EscalationId	INT	Primary Key, Identity	Unique escalation identifier
IncidentId	INT	NOT NULL, Foreign Key	Incident being escalated
EscalatedBy	INT	NULL, Foreign Key	User who escalated
EscalatedTo	INT	NULL, Foreign Key	User escalated to
Reason	VARCHAR(500)	NOT NULL	Escalation reason
EscalatedAt	DATETIME	NOT NULL, Default GETDATE()	Escalation date/time
ResolvedAt	DATETIME	NULL	Escalation resolution date/time

Foreign Keys

`Escalations.IncidentId → Incidents.IncidentId`

`Escalations.EscalatedBy → Users.UserId`

`Escalations.EscalatedTo → Users.UserId`

Purpose

This table supports escalation of incidents that require additional attention.

The Requirement Analysis requires the IT Manager to monitor escalated incidents and the system to support automated escalation when defined conditions are met.

6. Relationship Summary

Relationship	Cardinality	Description
Roles → Users	1 : Many	One role can be assigned to multiple users
Users → Incidents	1 : Many	One user can create multiple incidents
Categories → Incidents	1 : Many	One category can contain multiple incidents
Incidents → IncidentAssignments	1 : Many	An incident can have assignment records
Users → IncidentAssignments	1 : Many	A user can be assigned multiple incidents
Incidents → IncidentComments	1 : Many	An incident can contain multiple comments
Users → IncidentComments	1 : Many	A user can create multiple comments
Incidents → AIAAnalysis	1 : Many	An incident can have AI analysis records
Incidents → Notifications	1 : Many	An incident can generate multiple notifications
Users → Notifications	1 : Many	A user can receive multiple notifications
Incidents → Escalations	1 : Many	An incident can have escalation records
Users → Escalations	1 : Many	A user can initiate / receive multiple escalations

7. Referential Integrity

Foreign keys are used to maintain relationships between related entities. Examples:

```

Users.RoleId          -> Roles.RoleId
Incidents.CreatedBy   -> Users.UserId
Incidents.CategoryId  -> Categories.CategoryId
IncidentAssignments.IncidentId -> Incidents.IncidentId
IncidentComments.IncidentId -> Incidents.IncidentId
AIAAnalysis.IncidentId -> Incidents.IncidentId

```

The use of foreign keys prevents records from referencing nonexistent parent records and helps maintain consistency within the database.

8. Initial SQL Structure

The following SQL represents the Version 1 database structure currently prepared by the team.

```
CREATE TABLE Roles
(
    RoleId INT IDENTITY(1,1) PRIMARY KEY,
    RoleName VARCHAR(50) NOT NULL
);

CREATE TABLE Users
(
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    RoleId INT NOT NULL,

    FOREIGN KEY (RoleId) REFERENCES Roles(RoleId)
);

CREATE TABLE Categories
(
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    CategoryName VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Incidents
(
    IncidentId INT IDENTITY(1,1) PRIMARY KEY,
    Title VARCHAR(200) NOT NULL,
    Description VARCHAR(MAX) NOT NULL,
    CategoryId INT NOT NULL,
    Priority VARCHAR(50) NOT NULL,
    Status VARCHAR(50) NOT NULL DEFAULT 'Open',
    CreatedBy INT NOT NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
    ResolvedAt DATETIME NULL,

    FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId),
    FOREIGN KEY (CreatedBy) REFERENCES Users(UserId)
);

CREATE TABLE IncidentAssignments
(
    AssignmentId INT IDENTITY(1,1) PRIMARY KEY,
    IncidentId INT NOT NULL,
    AssignedTo INT NOT NULL,
    AssignedAt DATETIME NOT NULL DEFAULT GETDATE(),

    FOREIGN KEY (IncidentId) REFERENCES Incidents(IncidentId),
    FOREIGN KEY (AssignedTo) REFERENCES Users(UserId)
);

CREATE TABLE IncidentComments
(
    CommentId INT IDENTITY(1,1) PRIMARY KEY,
```

```

    IncidentId INT NOT NULL,
    UserId INT NOT NULL,
    CommentText VARCHAR(MAX) NOT NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),

    FOREIGN KEY (IncidentId) REFERENCES Incidents(IncidentId),
    FOREIGN KEY (UserId) REFERENCES Users(UserId)
);

CREATE TABLE AIAAnalysis
(
    AIAAnalysisId INT IDENTITY(1,1) PRIMARY KEY,
    IncidentId INT NOT NULL,
    SuggestedCategory VARCHAR(100),
    SuggestedPriority VARCHAR(50),
    SuggestedResolution VARCHAR(MAX),
    RelatedIncidentId INT NULL,
    ConfidenceScore DECIMAL(5,2),
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),

    FOREIGN KEY (IncidentId) REFERENCES Incidents(IncidentId),
    FOREIGN KEY (RelatedIncidentId) REFERENCES Incidents(IncidentId)
);

CREATE TABLE Notifications
(
    NotificationId INT IDENTITY(1,1) PRIMARY KEY,
    UserId INT NOT NULL,
    IncidentId INT NULL,
    Message VARCHAR(500) NOT NULL,
    IsRead BIT NOT NULL DEFAULT 0,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),

    FOREIGN KEY (UserId) REFERENCES Users(UserId),
    FOREIGN KEY (IncidentId) REFERENCES Incidents(IncidentId)
);

CREATE TABLE Escalations
(
    EscalationId INT IDENTITY(1,1) PRIMARY KEY,
    IncidentId INT NOT NULL,
    EscalatedBy INT NULL,
    EscalatedTo INT NULL,
    Reason VARCHAR(500) NOT NULL,
    EscalatedAt DATETIME NOT NULL DEFAULT GETDATE(),
    ResolvedAt DATETIME NULL,

    FOREIGN KEY (IncidentId) REFERENCES Incidents(IncidentId),
    FOREIGN KEY (EscalatedBy) REFERENCES Users(UserId),
    FOREIGN KEY (EscalatedTo) REFERENCES Users(UserId)
);

```

9. Pending Items for Database Design

The Version 1 structure covers the major core entities, but database design is not yet final. The following items remain pending and should be reviewed before Version 2 / final database design.

9.1 Incident Attachments

The Requirement Analysis states that employees should be able to attach relevant supporting information to an incident (FR-03).

Current status: Not represented in Version 1.

Pending decision: Determine whether a separate IncidentAttachments table is required and how files will be stored.

9.2 Incident Status History

The current Incidents table stores only the current status. However, the Requirement Analysis expects users to view incident history and requires incident status changes to be recorded consistently.

Current status: Not represented separately in Version 1.

Pending decision: Determine whether an IncidentStatusHistory table should be introduced.

9.3 Employee Feedback

FR-08 requires employees to provide feedback after an incident is resolved.

Current status: Not represented in Version 1.

Pending decision: Determine the feedback fields, rating system, and whether one or multiple feedback submissions should be allowed.

9.4 Audit Logs

The Requirement Analysis requires administrators to view relevant system activity logs and states that important user and system actions should be auditable.

Current status: Not represented in Version 1.

Pending decision: Determine which actions require auditing and whether a dedicated AuditLogs table will be used.

9.5 System Configuration

FR-21 requires administrators to manage system configuration settings.

Current status: Not represented in Version 1.

Pending decision: Determine which configuration values need to be stored in the database and whether a SystemConfiguration table is required.

9.6 AI Conversation Summary

The Requirement Analysis requires the system to generate summaries of lengthy incident conversations. The current AIAalysis table stores category, priority, resolution, related incident, and confidence score, but does not explicitly contain a conversation-summary field.

Current status: Pending database mapping.

Pending decision: Determine whether ConversationSummary should be added to AIAalysis or stored in a separate entity.

9.7 Resolution / Investigation Details

The Requirement Analysis states that Help Desk Agents should be able to record investigation and resolution details. The current Version 1 structure contains ResolvedAt, but does not explicitly store investigation notes or final resolution details.

Current status: Pending database mapping.

Pending decision: Determine whether resolution information should be stored in Incidents, IncidentComments, or a separate resolution-related table.

9.8 Incident Type

The Requirement Analysis uses the term “IT incident or service request.” The current Incidents table does not contain a field that distinguishes an incident from a service request.

Current status: Pending clarification.

Pending decision: Determine whether the system will treat both as the same entity or introduce an IncidentType field.

9.9 Priority and Status Values

The current database stores Priority and Status as VARCHAR.

Current status: Under review.

Pending decision: The team needs to finalize the allowed values, for example:

Priority	Status
Low	Open
Medium	Assigned
High	In Progress
Critical	Resolved
	Closed

The exact values should be finalized during System Design.

9.10 User Deactivation

The Requirement Analysis requires administrators to create, update, and deactivate user accounts. The current Users table does not contain an explicit account-status or active/inactive field.

Current status: Pending.

Pending decision: Determine whether an IsActive field or equivalent account-status field is required.

10. Database Design Validation Pending

Before the database is considered final, the team should validate the following:

- ☐ Every Functional Requirement has corresponding database support where required.
- ☐ Every AI requirement has corresponding data storage where required.
- ☐ Automation requirements have appropriate database support.
- ☐ Incident lifecycle and status history are properly represented.
- ☐ Assignment and reassignment behavior is finalized.
- ☐ Resolution information storage is finalized.
- ☐ Attachments are finalized.
- ☐ Feedback is finalized.
- ☐ Audit logging is finalized.
- ☐ System configuration storage is finalized.
- ☐ Incident vs. service-request handling is finalized.
- ☐ Priority values are finalized.
- ☐ Status values are finalized.
- ☐ User account deactivation is finalized.
- ☐ Primary keys and foreign keys are validated.
- ☐ Required fields and NULL / NOT NULL constraints are validated.
- ☐ Indexes are reviewed based on expected queries.
- ☐ Delete / update behavior for foreign-key relationships is reviewed.
- ☐ Database design is reviewed against the final System Design.

11. Version 1 Status

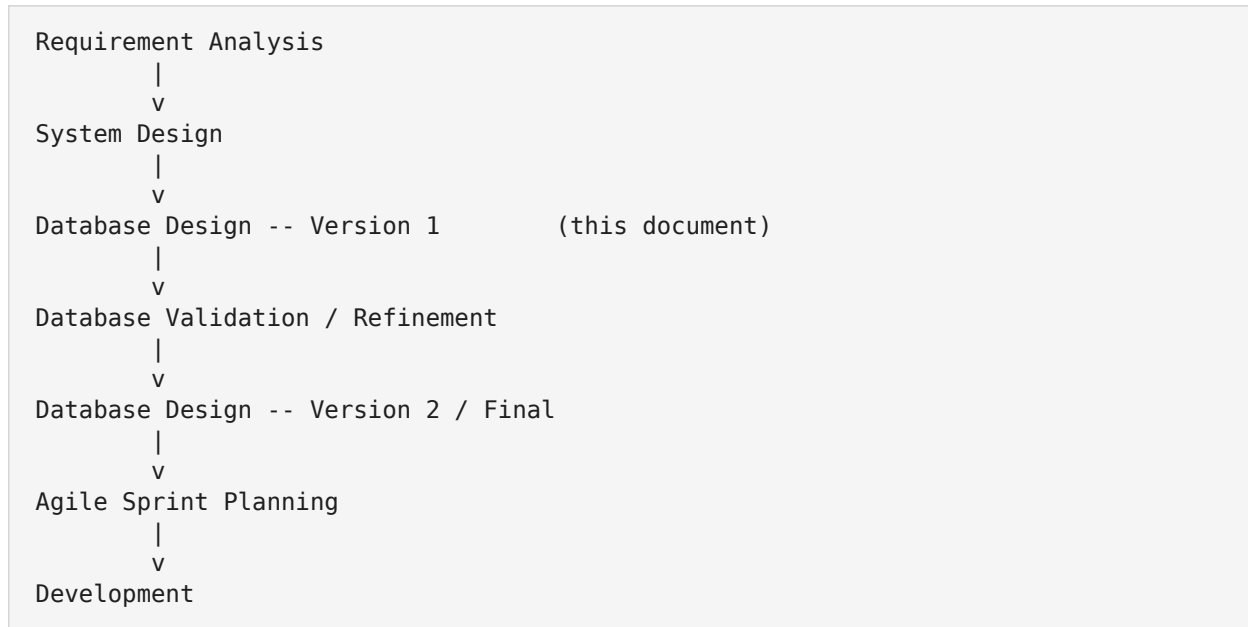
Item	Value
Database Version	1.0
Status	Initial Structure / Under Review
Tables	9
Core Structure	Defined
Relationships	Defined
SQL Structure	Prepared
Requirement Mapping	Partially validated
Finalization	Pending System Design and database validation

12. Next Step

The Version 1 database structure provides the initial foundation for the project.

The next step is to review the database against the System Design and Requirement Analysis and determine which pending items should be included in Database Version 2.0.

Overall Project Flow



This approach ensures that the database is not treated as an isolated design, but remains traceable to the requirements and later development work. The Requirement Analysis specifically establishes this traceability from business requirement through database design and development.

13. Conclusion

Version 1 establishes the core relational database structure for the AI-Powered IT Service Desk & Incident Management System.

The current structure contains the major entities required for users, roles, categories, incidents, assignments, communication, AI analysis, notifications, and escalations.

Several requirements identified during Requirement Analysis still require explicit database mapping. These items are intentionally marked as pending rather than being added without team validation.

The Version 1 database therefore serves as the initial database baseline for the project. Further refinement will be performed after System Design and detailed requirement-to-database validation.